GUI Based Multi-Client Chat Application Using TCP Python Socket Programming with Tkinter

Student Details

Name: K MOHANA SWAROOP

Roll Number: 323506402262

Course: Computer Networks Laboratory

Assignment: Assignment 5 – Advanced Multi-Client Chat Server using TCP

Programming Language: Python 3

Operating System: Ubuntu 24.04 LTS

Virtualization Platform: Oracle VirtualBox

1. Objective

The objective of this assignment is to convert the terminal-based multi-client chat application developed in Assignment 5 into a graphical desktop application using Python Tkinter. The existing TCP server is reused while replacing the terminal client with a user friendly graphical interface. The application demonstrates GUI programming, event driven programming, multithreading, socket programming, and client-server communication.

2. GUI Design

The application consists of two windows.

Login Window:

• Server IP Address

• Username

• Optional Password

• Connect Button

Main Chat Window:

• Scrollable chat area

• Message entry box

• Send button

• Disconnect button

• Online users panel

• Connection status label

The layout is organized using Frames. ScrolledText is used for displaying messages,

Entry widgets accept user input, and Listbox displays online users.

3. System Architecture

Architecture:

GUI Client (Tkinter)

|

TCP Socket Communication

|

Assignment 5 Server

|

CSV Chat History

The server accepts multiple client connections, authenticates usernames, receives

messages, broadcasts messages, supports private messaging, updates online users, stores

chat history in CSV, and handles disconnections.

4. Components Reused from Assignment 5

The following modules from Assignment 5 were reused without major modifications:

• TCP Socket Communication

• Multi-client server implementation

• Broadcast messaging

• Private messaging using /msg command

• Online user management

• Chat history storage in CSV

• Login and logout notifications

• Server statistics

• Thread-based client handling

Reusing the previous server reduced development time and ensured compatibility with

the networking logic.

5. GUI Design Decisions

Tkinter was selected because it is included with Python and provides sufficient widgets

for desktop applications.

Design choices:

• Frames separate chat controls from online users.

• ScrolledText provides automatic scrolling for chat history.

• Entry widget allows easy message typing.

• Buttons simplify user interaction.

• Listbox displays currently connected users.

• Message boxes provide success and error dialogs.

• A status label displays connection status.

• Event binding allows pressing Enter to send messages.

6. Testing Results

Testing was performed using Mininet with one server and multiple clients.

Tests performed:

1. Successful client login.

2. Multiple client connections.

3. Broadcast messaging.

4. Private messaging.

5. Online user list updates.

6. Join notifications.

7. Leave notifications.

8. Disconnect handling.

9. Chat history display.

10. GUI responsiveness during message reception.

Result: All required functionalities were successfully verified.

7. Wireshark Verification

Wireshark filter:

tcp.port == 5000

Observed packets:

• TCP three-way handshake.

• Client login packets.

• Broadcast message packets.

• Private message packets.

• Client disconnect packets.

Insert screenshots with brief explanations for each capture.

8. Reflection Answers

Why separate networking and GUI?

Separating networking from the GUI improves maintainability, modularity, debugging,

and code reuse.

Why is a background thread required?

Socket receive() blocks execution. Running it in a background thread keeps the GUI

responsive while messages arrive.

Which Assignment 5 components were reused?

The socket communication layer, server implementation, broadcast logic, private messaging, CSV logging, online users, and chat history.

How did the GUI improve usability?

The graphical interface is easier to use, displays messages clearly, shows online users, supports buttons, and improves the overall user experience.

Future enhancement:

Support file transfer, emojis, encryption, authentication, voice chat, and message timestamps.

9. Conclusion

The Assignment 6 objectives were successfully achieved. The terminal client was replaced by a Tkinter-based graphical client while reusing the Assignment 5 server. The final application supports multiple users, broadcast messaging, private messaging, online user management, chat history, and a responsive graphical interface. The project demonstrates practical implementation of TCP socket programming, multithreading, and GUI development.

Appendix A - Software Requirements

- Ubuntu Linux
- Python 3
- Tkinter
- Socket Library
- Threading Module
- CSV Module
- Mininet
- Wireshark
- VirtualBox.